A New Framework for Efficient Multivariate Functional Bootstrapping

Kunyu Wu[1], Kuiyuan Duan[1], Dengfa Liu[1], and Hongbo Li[⋆1]

Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China.
E-mail: wukunyu@amss.ac.cn,dkuei@outlook.com, dengfa@amss.ac.cn,hli@mmrc.iss.ac.cn

Abstract. Fully homomorphic encryption (FHE) enables computation on encrypted data without decryption. In TFHE, programmable bootstrapping (PBS) evaluates nonlinear functions through lookup tables (LUTs), but a direct multivariate LUT over a t-ary plaintext space has size t^ℓ. This paper studies LUT compression for multivariate functional bootstrapping via variable separation and additive inner representations. We first apply this approach to non-negative integer division with remainder. For a dividend m, a divisor d, and $h = \lfloor m/d \rfloor$, we use a logarithmic transformation to decompose bivariate division into two univariate logarithmic PBS calls, one homomorphic subtraction, and one outer exponential PBS call. To handle integer plaintexts, we introduce a rounded logarithmic function $\mathrm{clog}_{B,M}$ and give a sufficient condition on M for exact quotient recovery. The resulting homomorphic division-with-remainder algorithm achieves $\widetilde{O}(1)$ equivalent blind-rotation complexity under theoretically optimal parameters, and also yields frameworks for modular reduction and truncated division.
We further prove that every finite function $f : [t]^\ell \to [t]$ can be written as

$$f(x_1, \ldots, x_\ell) = q\left(\sum_{i=1}^{\ell} p_i(x_i)\right),$$

and search for small-span representations using simulated annealing with reheating. Experiments show a 3.6× speedup for division with remainder at $t = 64$, and a 1.9× speedup for the Hamming-weight interval function, compared with estimates based on [3].

Keywords: Fully Homomorphic Encryption · Programmable Bootstrapping · Functional Bootstrapping · LUT Compression · Integer Division.

1 Introduction

Fully homomorphic encryption (FHE) is a cryptographic primitive that supports computation directly on encrypted data. The first feasible construction was proposed by Gentry [15]. Later, second-generation schemes such as BGV [4] and

⋆ Corresponding author.

B/FV [5, 13] were introduced, and HElib [16] provided one of the first practical FHE libraries based on this line of schemes. Another widely used family is CKKS [11], which is designed for approximate arithmetic. In this work, we focus on exact integer arithmetic over finite plaintext sets and mainly use the TFHE/FHEW line of schemes [12, 8].

We write $[t] = \{0, 1, \ldots, t-1\}$ for the set of residues represented by a t-ary plaintext space. FHE schemes can efficiently support additions, subtractions, and multiplications on ciphertexts, either natively or through standard homomorphic procedures. Division with remainder is nevertheless a fundamental operation: many integer programs, including modular reduction, normalization, and fixed-point scaling, depend on the quotient or the remainder of two values. When both the dividend and the divisor are encrypted, the divisor cannot be used to select a public circuit branch or a public lookup-table entry. Ciphertext-by-ciphertext division therefore becomes a substantially harder primitive than ordinary homomorphic arithmetic.

We view division with remainder as a bivariate function over a finite plaintext set. This perspective leads us to study multivariate functional bootstrapping for functions with decomposable lookup-table structure. Instead of evaluating one large multivariate LUT directly, the goal is to represent it using smaller univariate LUTs together with a limited number of homomorphic linear operations and ciphertext operations. Efficient processing of such decompositions provides a useful primitive for compiling homomorphic integer programs and for evaluating nonlinear functions over encrypted data.

1.1 Related Work

Homomorphic Division There are several lines of research on homomorphic division. One line focuses on approximate division, typically in floating-point arithmetic, which is orthogonal to our goal of exact integer division. Prantl et al. [22] recently studied efficient approximate division under CKKS. Another mature line considers the simpler but practical setting of ciphertext-by-plaintext division, where the divisor is known in plaintext. Earlier efficient methods often required the plaintext divisor to be a multiple of small constants such as 4. The state-of-the-art result by Akherati et al. [1] removes this restriction and achieves high-throughput division by arbitrary plaintext divisors. These methods are valuable, but do not directly apply to ciphertext-by-ciphertext integer division, where both operands are encrypted and an exact integer result is required.

Early methods for integer division [7, 25, 6] were built on bit-wise encryption schemes based on BGV [4] and B/FV [5, 13]. However, carry propagation makes large-integer division impractical. To overcome this, Okada et al. [20] introduced an integer-wise division algorithm for BGV using polynomial interpolation and equality-check functions. Later, Okada et al. [21] proposed a more efficient integer-wise method based on TFHE, reducing complexity to $O(t)$ TFHE bootstrapping LUT calls (where t is the plaintext modulus), achieving a square-root improvement over [20]. More recently, [18] and [19] achieved constant-factor

speedups and supported larger plaintext integers; in particular, [19] proposed a slot-based LUT technique under BGV, enabling 15-bit division in roughly 300 seconds.

In contrast, our method reduces complexity to $\widetilde{O}(1)$ bootstrapping operations—an exponential improvement over the $O(t)$ complexity of [21]. Prior works enumerate all possible divisors and use the divisor ciphertext to select the correct quotient, introducing larger error and possibly requiring additional bootstrapping. Our algorithm requires only TFHE bootstrapping and a constant number of FHE additions, achieving theoretically optimal error. Combining [19] with our method for faster, lower-error division on larger plaintexts is a promising future direction.

Decomposition and Optimization of Large Lookup Tables The work most directly related to this paper is the large-LUT decomposition method of Belaid, Bon, and Rivain [3]. Before [3], several works had improved large-LUT evaluation in TFHE. The tree-based method (TBM) of [14] splits the input message into several low-precision ciphertexts and evaluates the target function through a multi-layer PBS tree. When combined with multi-value bootstrapping [10], the most expensive first layer can be parallelized, making the method suitable for 8-bit LUTs such as AES S-boxes [24]. Another representative approach is WoP-PBS [2], which uses circuit bootstrapping to transform LWE ciphertexts into GGSW ciphertexts, extracts message bits as encrypted selectors, evaluates a CMUX tree, and finally uses a classical PBS to select the output.

These methods achieve important improvements, but they rely on specialized homomorphic operators and follow a different optimization strategy. Belaid et al. [3] instead proposed a decomposition framework based only on standard TFHE PBS. For a target function $f(x_1, \ldots, x_\ell)$ with $x_i \in [t]$, whose direct LUT representation has size t^n, their method expresses the function using auxiliary nonlinear values generated by small LUTs, linear combinations, and a small number of products. The unknown coefficients of this representation are obtained by solving a linear system, and the resulting decomposition is then evaluated homomorphically with a reduced number of small PBS calls. A brief description of this framework is given in Appendix A.

1.2 Our Contribution

This paper makes the following three main contributions.

A more efficient algorithm for non-negative division with remainder. Our algorithm follows the idea of variable separation. For a dividend m and a nonzero divisor d, observe that

$$\frac{m}{d} = 2^{\log m - \log d}.$$

Thus, intuitively, one can first table-evaluate $\log m$ and $\log d$, compute their difference, and finally table-evaluate $\lfloor 2^{\log m - \log d} \rfloor$.

However, the input and output values in TFHE functional bootstrapping must be integers. Fortunately, we can use scaling to control precision. We prove the following theorem, where clog is an extended rounded-logarithm function formally defined in Definition 1.

Theorem 1. *For any integer $m \in [t]$, any nonzero divisor $d \in [t] \setminus \{0\}$, any scaling parameter M satisfying*

$$M \geq \frac{2}{\log_B(1 + \frac{1}{t-1})},$$

and any base $B > 1$, the quotient h of m divided by d can be computed as

$$h = \lfloor B^{\delta_M / M} \rfloor,$$

where

$$\delta_M = \mathrm{clog}_{B,M}(m) - \lfloor M \log_B d \rfloor.$$

This shows that our method only needs three calls to the general functional bootstrapping procedure (GFBP), evaluating $\mathrm{clog}_{B,M}$, $\lfloor M \log_B(\cdot) \rfloor$, and $\lfloor B^{\cdot/M} \rfloor$, respectively. The resulting complexity is $\widetilde{O}(1)$ functional bootstrappings.

Applications of non-negative division with remainder. Based on the above non-negative integer division algorithm, we further give homomorphic frameworks for several common integer operations. First, modular reduction $m \bmod d$ is accelerated accordingly. Second, we extend non-negative division with remainder to truncated division over general integers. The core idea is to use functional bootstrapping to obtain the sign bits of the dividend and divisor, and then obtain ciphertexts of $|m|$ and $|d|$. We invoke the non-negative division algorithm to compute the non-negative quotient $\lfloor |m|/|d| \rfloor$, and finally recover the sign of the truncated quotient from the XOR relation between the sign bits of the dividend and divisor. Hence, general integer truncated division is reduced to the composition of sign processing, absolute-value computation, and non-negative division with remainder.

Efficient functional bootstrapping for general multivariate functions. For a general multivariate function $f : [t]^n \to [t]$, we propose a universal bootstrapping framework based on additive decomposition into univariate functions:

$$f(x_1, \ldots, x_\ell) = q\left(\sum_{i=1}^{n} p_i(x_i)\right),$$

where each $p_i : [t] \to \mathbb{Z}$ is a univariate inner function and q is the outer function defined on the resulting sum values. We prove that such a representation always exists for every multivariate function over a finite domain. Moreover, when the output classes of f can be separated by relatively few sum values, the domain of the outer LUT can be much smaller than the original input space of size t^n.

To find a small-span additive representation, we use simulated annealing with reheating to search for the inner functions p_i, thereby compressing the outer LUT. Experimental results show that for multivariate functions with statistical aggregation structure, such as mode, median, general symbol-set threshold functions, and interval functions, our method reduces the total number of inner and outer LUT records and outperforms the large-LUT decomposition method of [3] in multiple parameter settings.

1.3 Organization

The rest of this paper is organized as follows. Section 2 recalls LWE/RLWE-based fully homomorphic encryption and TFHE functional bootstrapping. Section 3 gives the efficient bootstrapping algorithm for division with remainder and analyzes its complexity and noise growth. Section 4 extends the scheme to multivariate functions. Section 5 presents experimental results. Section 6 concludes the paper.

2 Preliminaries

2.1 Notation

$A \backslash B$ denotes the set obtained from A by removing all elements that belong to B. E^p denotes the set of p-dimensional vectors over E. For a positive integer t, define the complete residue system modulo t as $\mathbb{Z}_t \coloneqq \{-\lfloor t/2 \rfloor, \ldots, 0, \ldots, \lfloor (t-1)/2 \rfloor\}$. For $x' \coloneqq x \bmod t$, we have $t \mid (x' - x)$ and $x' \in \mathbb{Z}_t$. Let $\bar{x}$ denote the element in $[0, t)$ satisfying $\bar{x} \equiv x \mod t$. The notation $[x]_t$ satisfies $[x]_t \equiv x \mod t$ and $[x]_t \in \mathbb{Z}_t$.

Let n, N be powers of two with $n < N$. Let $q \leq q'$ be positive integers. For $M \in \{n, N\}$ and $p \in \{q, q'\}$, define

$$\mathcal{R}_M \coloneqq \mathbb{Z}[x]/(x^M + 1), \qquad \mathcal{R}_{M,p} \coloneqq \mathbb{Z}_p[x]/(x^M + 1).$$

We use $|\cdot|$ for absolute value, $\lceil \cdot \rceil$ for ceiling, and $\lfloor \cdot \rfloor$ for floor. The symbol $\otimes$ denotes tensor product.

For any integer x, define

$$\mathrm{sign}(x) = \begin{cases} 1, & x \geq 0, \\ -1, & x < 0. \end{cases}$$

Thus $x = \mathrm{sign}(x)|x|$ for all $x \in \mathbb{Z}$. For any integer x, define

$$\mathrm{msb}(x) = \begin{cases} 0, & x \geq 0, \\ 1, & x < 0. \end{cases}$$

For a subset $A \subseteq [t]$, let $c_A(\mathbf{x})$ be the number of coordinates of $\mathbf{x} \in [t]^n$ whose values lie in A, namely

$$c_A(\mathbf{x}) = \#\{i \in [n] : x_i \in A\}.$$

When $A = \{a\}$, we write $c_a(\mathbf{x})$. In the binary case [2], the Hamming weight is $w_H(\mathbf{x}) = c_1(\mathbf{x})$. If the mode is not unique, we choose the smallest element in $[t]$. The median is taken in the natural order $0 < 1 < \cdots < t-1$, and the lower median is used when n is even.

2.2 LWE Encryption

The Learning with Errors (LWE) cryptosystem [23] uses a security parameter λ, plaintext modulus t, ciphertext modulus q, lattice dimension $n := n(\lambda)$, and error distribution $\chi := \chi(\lambda)$. Let $\mathbf{s} \in \mathbb{Z}^n$ be the secret key. An LWE encryption of $m \in \mathbb{Z}_t$ under $\mathbf{s}$, denoted by $\mathrm{LWE}_{\mathbf{s},q/t}(m)$, is a pair $(\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$ satisfying

$$b = \langle \mathbf{a}, \mathbf{s} \rangle + m\lfloor q/t \rfloor + e \mod q,$$

where $e \leftarrow \chi$ is a small error with $2|e| < \lfloor q/t \rfloor$. To decrypt, compute

$$\widetilde{m} = \frac{t}{q} \cdot (b - \langle \mathbf{a}, \mathbf{s} \rangle) \bmod q,$$

and round $\widetilde{m}$ to the nearest element of $\mathbb{Z}_t$.

2.3 TFHE Functional Bootstrapping

Negacyclic functions. Let t be a positive even integer. A function $f : \mathbb{Z}_t \to \mathbb{Z}_{t'}$ is called negacyclic if for every $k \in [t/2]$,

$$f(k + t/2 \bmod t) = -f(k) \bmod t'.$$

Let $N > t$, and let $f' : \mathbb{Z}_{2N} \to \mathbb{Z}_{t'}$ be a function. If for all $m \in \mathbb{Z}_t$ and all $e \in \mathbb{Z}_{2N}$ satisfying $2|e| < \lfloor 2N/t \rfloor$, we have

$$f'(m\lfloor 2N/t \rfloor + e) = f(m),$$

then f' is called a negacyclic extension of f from $\mathbb{Z}_t$ to $\mathbb{Z}_{2N}$.

Negacyclic functional bootstrapping. Given a ciphertext $\mathrm{LWE}_{\mathbf{s},q/t}(m)$ encrypting $m \in \mathbb{Z}_t$ and a negacyclic function $f : \mathbb{Z}_t \to \mathbb{Z}_{t'}$, negacyclic functional bootstrapping produces

$$\mathrm{FBP} : \mathbb{Z}_q^{n+1} \times \mathcal{F}_{t,t'} \longrightarrow \mathbb{Z}_{q'}^{n+1}, \qquad (\mathrm{LWE}_{\mathbf{s},q/t}(m), f) \longmapsto \mathrm{LWE}_{\mathbf{s},q'/t'}(f(m)).$$

It outputs a decryptable ciphertext under the same secret key encrypting a message in $\mathbb{Z}_{t'}$.

GFBP. Similarly, given $\mathrm{ct} \in \mathrm{LWE}_{\mathbf{s},q/t}(m)$ with $m \in \mathbb{Z}_t$ and an arbitrary function $f : \mathbb{Z}_t \to \mathbb{Z}_{t'}$, the general functional bootstrapping procedure (GFBP) outputs

$$\mathrm{GFBP} : \mathbb{Z}_q^{n+1} \times \mathcal{F}_{t,t'} \longrightarrow \mathbb{Z}_q^{n+1}, \qquad (\mathrm{LWE}_{s,q/t}(m), f) \longmapsto \mathrm{LWE}_{\mathbf{s},q/t'}(f(m)).$$

The implementation of GFBP is more complex than FBP; see [26]. For convenience, following [26], we assume $q = 2N$, where N is the polynomial ring degree used in functional bootstrapping.

In the subsequent theoretical complexity analysis, we assume that one invocation of FBP on $(\mathrm{LWE}_{\mathbf{s},q/(2t)}(m), f)$ costs one $\mathbb{Z}[\zeta_q]$-BR, namely one blind rotation over the cyclotomic ring, and produces final noise ν_{BR}. The method of [26] shows that one GFBP invocation on $(\mathrm{LWE}_{\mathbf{s},q/(2t)}(m), f)$ costs one $\mathbb{Z}[\zeta_q]$-BR and introduces noise growth $O(\omega^4 N^7)\nu_{\mathrm{BR}}$, where ω is the Hamming weight of the secret key. Alternatively, to obtain smaller noise ν_{BR}, one may use the method of [17] in the necessary steps, at cost $2 \times \mathbb{Z}[\zeta_q]$-BR. Since the cost is proportional to $\mathbb{Z}[\zeta_q]$-BR and this term depends on q, the computational cost grows linearly with the modulus q.

Table 1. Comparison of functional bootstrapping methods.

Work	Function	Noise growth	Computational cost
[9]	FBP	ν_{BR}	$1 \times \mathbb{Z}[\zeta_q]$-BR
[17]	GFDP	ν_{BR}	$2 \times \mathbb{Z}[\zeta_q]$-BR
[26]	GFBP	$O(\omega^4 N^7)\nu_{\mathrm{BR}}$	$1 \times \mathbb{Z}[\zeta_q]$-BR

2.4 Simulated Annealing with Reheating

Simulated annealing is an optimization algorithm used to find an optimal or near-optimal solution in a search space, namely a solution with small energy E.

Similar to classical simulated annealing, we use simulated annealing with reheating: when the energy does not improve for a long period, the temperature is increased again.

The procedure is as follows:

1. Initialize a feasible solution with energy E, initial temperature T_0, and cooling rate c.
2. Search in the neighborhood of the current solution. If the neighboring solution satisfies the constraints, compute $\Delta E = E_{\mathrm{new}} - E_{\mathrm{old}}$. If $\Delta E < 0$, accept the new solution. If $\Delta E > 0$, accept it with probability $\exp(-\Delta E/T)$.
3. After each iteration, reduce the temperature by $T_{\mathrm{new}} = T_{\mathrm{old}} \cdot c$.
4. If E does not improve after many iterations, assume that the search is trapped in a local optimum and reheat to increase the probability of accepting worse solutions.
5. When the iteration limit is reached, output the final energy and the corresponding solution.

3 Efficient Bootstrapping for Explicitly Variable-Separable Multivariate Functions

Our goal is to efficiently bootstrap multivariate functions $f(x_1, x_2, \ldots, x_\ell)$. For simplicity, we assume below that for every $i \in [\ell]$, $x_i \in [t]$ and $f(x_1, \ldots, x_\ell) \in [t]$. It is straightforward to extend the arguments to the case $x_i \in [t_i]$ and $f(x_1, \ldots, x_\ell) \in [t']$, where the plaintext alphabet sizes t_i and t' may differ.

In this section, we first consider a special class of functions for which $f(x_1, \ldots, x_\ell)$ can be easily decomposed into an additive combination of univariate functions $\{g_i(x_i)\}$, and these univariate functions have explicit expressions.

We use non-negative integer division with remainder as a practical example to explain how elementary transformations separate variables and compress lookup tables, thereby accelerating multivariate functional bootstrapping.

3.1 Efficient Bootstrapping for Non-negative Integer Division

We observe that

$$\frac{m}{d} = 2^{\log m - \log d}.$$

Thus, one may first table-evaluate $\log m$ and $\log d$, then compute their difference, and finally table-evaluate $\lfloor 2^{\log m - \log d} \rfloor$.

However, in TFHE functional bootstrapping, both the input and output of the table lookup must be integers, while $\log m$ and $\log d$ are generally not integers. We therefore scale $\log m$ and $\log d$ by a factor M, depending on t, and use floor or ceiling rounding to control precision. The outer function $\lfloor 2^{(\cdot)} \rfloor$ is replaced by $\lfloor 2^{(\cdot/M)} \rfloor$.

There are two points to justify. First, such an M indeed exists so that the precision control is effective and the final output is exactly $\lfloor 2^{\log m - \log d} \rfloor$. This is proved below. Second, the resulting table size is indeed smaller. The inner tables for $\lceil M \log m \rceil$ and $\lfloor M \log d \rfloor$ each have size t. For the outer table, an appropriate choice of M makes its size $O(t \log t)$. Intuitively, inputs such as $(2, 1), (4, 2), (6, 3), \ldots$ would be stored separately in a direct table, but under the logarithmic-difference framework they map to the same outer input value and are compressed into one record.

Framework for Non-negative Division with Remainder We first consider the plaintext setting. We seek M such that

$$\left\lfloor \frac{m}{d} \right\rfloor = \left\lfloor 2^{\lceil M \log m \rceil - \lfloor M \log d \rfloor} \right\rfloor$$

holds for all $m, d \in [t] \setminus \{0\}$. The framework is not limited to logarithm base 2; it applies to any base $B > 1$. If $m = 0$, replacing $\lceil M \log m \rceil$ by any negative integer such as -1 gives the correct division result. This motivates the following definition.

Definition 1. *For any $B > 1$ and positive number M, the (B, M)-ceil logarithmic function is*

$$\mathrm{clog}_{B,M} : \mathbb{N} \to \mathbb{Z}, \qquad \alpha \mapsto \begin{cases} \lceil M \log_B \alpha \rceil, & \alpha > 0, \\ -1, & \alpha = 0. \end{cases}$$

Combining the clog function with the above analysis gives the non-negative division algorithm.

Algorithm 1: Homomorphic integer division $\mathrm{IntDiv}(\mathrm{ct}_m, \mathrm{ct}_d)$

Input: $\mathrm{ct}_m \in \mathrm{LWE}_{\mathbf{s},q/t}(m)$, $\mathrm{ct}_d \in \mathrm{LWE}_{s,q/t}(d)$; bootstrapping key BK, key-switching key KS

Output: $\mathrm{ct}_h \in \mathrm{LWE}_{\mathbf{s},q/(2t)}(h)$, where $h \in \mathbb{Z}_{2t}$, $m = hd + r$, and $r \in [d]$

$\mathrm{ct}_{tm} \in \mathrm{LWE}_{\mathbf{s},q'/(2t')}(\mathrm{clog}_{B,M}(m)) \leftarrow \mathrm{GFBP}(\mathrm{ct}_m, \mathrm{clog}_{B,M})$;

$\mathrm{ct}_{td} \in \mathrm{LWE}_{\mathbf{s},q'/(2t')}(\lfloor M \log_B(d) \rfloor) \leftarrow \mathrm{GFBP}(\mathrm{ct}_d, \lfloor M \log_B(\cdot) \rfloor)$;

$\mathrm{ct}_\delta = \mathrm{ct}_m - \mathrm{ct}_d$// $\mathrm{ct}_\delta \in \mathrm{LWE}_{\mathbf{s},q'/(2t')}(\delta)$

return $\mathrm{ct}_h = \mathrm{GFBP}(\mathrm{ct}_\delta, \lfloor B^{\cdot/M} \rfloor) \in \mathrm{LWE}_{\mathbf{s},q/(2t)}(h)$;

The parameters B and M may take other values, not necessarily integers, as long as they satisfy the theorem below.

Theorem 2. *Given any integer $m \in [t]$ and positive integer $d \in [t] \setminus \{0\}$, for any positive number $M \geq 2/\log_B(1 + \frac{1}{t-1})$ and $B > 1$, the quotient h of d divides m can be computed by*

$$h = \lfloor B^{\delta_M / M} \rfloor \tag{3.1}$$

where

$$\delta_M = \mathrm{clog}_{B,M}(m) - \lfloor M \log_B d \rfloor. \tag{3.2}$$

Proof. Suppose $r \in [d]$ is the remainder of d divides m so that $m = hd + r$.

If $m \neq 0$, there is $e_m = \lceil M \log_B m \rceil - M \log_B m \in [0, 1)$ and $e_d = M \log_B d - \lfloor M \log_B d \rfloor \in [0, 1)$ so that

$$\delta_M = (M \log_B m + e_m) - (M \log_B d - e_d) = M \log_B(m/d) + (e_m + e_d). \tag{3.3}$$

Let $e = e_m + e_d \in [0, 2)$, then

$$\lfloor B^{\delta_M / M} \rfloor = \lfloor m/d \cdot B^{e/M} \rfloor = h + \lfloor m/d \cdot B^{e/M} - h \rfloor. \tag{3.4}$$

For any positive M, it is obvious that

$$m/d \cdot B^{e/M} - h \geq m/d - h = r/d \geq 0. \tag{3.5}$$

On the contrary, since $M \geq 2/\log_B(1 + \frac{1}{t-1})$,

$$\begin{aligned} m/d \cdot B^{e/M} - h &\leq m/d \cdot B^{e \log_B(1 + \frac{1}{t-1})/2} - h \\ &= m/d \cdot (\frac{t}{t-1})^{e/2} - h \\ &< m/d \cdot \frac{t}{t-1} - h. \end{aligned} \tag{3.6}$$

Due to $\frac{t}{t-1} = 1 + \frac{1}{t-1}$ is a monotonically decreasing function and $m \leq t-1$, thus

$$m/d \cdot B^{e/M} - h < m/d \cdot (m+1)m - h = (r+1)/d \leq 1. \tag{3.7}$$

Combining (3.4),(3.5) and (3.7), $\lfloor B^{\delta_M/M} \rfloor = h$.

If $m = 0$, then $h = 0$ and there is $e_d = M \log_B d - \lfloor M \log_B d \rfloor \in [0, 1)$ so that

$$\delta_M = -1 - M \log_B d + e_d. \tag{3.8}$$

Furthermore, for any positive integer M,

$$B^{\delta_M/M} = B^{(e_d-1)/M}/d < 1/d \leq 1. \tag{3.9}$$

Thus, $\lfloor B^{\delta_M/M} \rfloor = 0 = h$.

The algorithm requires $B > 1$, $M \geq 2/\log_B(1 + \frac{1}{t-1})$, and a positive integer $t' \geq \lfloor M \log_B(t-1) \rfloor + 1$. We restrict the ranges of $\mathrm{clog}_{B,M}$ and $\lfloor M \log_B(\cdot) \rfloor$ to $\mathbb{Z}_{2t'}$.

By Theorem 2 and the correctness of GFBP, Algorithm 1 is correct. Its noise comes from the final GFBP call implemented using [17], and is ν_{BR}.

Complexity analysis. To balance complexity and noise growth, Step 1 and Step 2 use the GFBP method of [26], while the final GFBP uses [17]. Since $m, d \in [t]$ and the ciphertext modulus is q, each of Step 1 and Step 2 costs $1 \times \mathbb{Z}[\zeta_q]$-BR.

The ranges of $\mathrm{clog}_{B,M}(m)$ and $\lfloor M \log_B(d) \rfloor$ are respectively

$$[-1, \lceil M \log_B(t-1) \rceil], \qquad [0, \lfloor M \log_B(t-1) \rfloor].$$

Thus the plaintext of ct_δ lies in

$$[-1 - \lfloor M \log_B(t-1) \rfloor,\ \lceil M \log_B(t-1) \rceil],$$

so the input plaintext space of the final GFBP is $\mathbb{Z}_{2t'+1}$, where $t' = \lfloor M \log_B(t-1) \rfloor + 1$.

If the noise may approach $\lfloor q/t \rfloor$, then after the plaintext modulus changes from t to $t'+1$, the ciphertext modulus must be scaled to at least

$$q' = \lceil q(\lfloor M \log_B(t-1) \rfloor + 2)/t \rceil$$

to guarantee correct LWE decryption. Hence the final GFBP costs

$$2 \times \mathbb{Z}[\zeta_{q'}]\text{-BR} \leq 2 \left\lceil \frac{\lfloor M \log_B(t-1) \rfloor + 2}{t} \right\rceil \times \mathbb{Z}[\zeta_q]\text{-BR}.$$

The total complexity is therefore

$$\left(2 + 2 \left\lceil \frac{\lfloor M \log_B(t-1) \rfloor + 2}{t} \right\rceil\right) \times \mathbb{Z}[\zeta_q]\text{-BR}.$$

To minimize the complexity for a fixed t, choose M, B so that

$$\frac{\lfloor M \log_B(t-1) \rfloor + 2}{t}$$

is as small as possible subject to $B > 1$ and $M \geq 2/\log_B(1+\frac{1}{t-1})$. The minimum possible value is $(\lfloor A \rfloor + 2)/t$, where

$$A = \frac{2\ln(t-1)}{\ln(1+\frac{1}{t-1})}.$$

It is achieved by choosing any $B > 1$ and setting

$$M = \frac{2}{\log_B(1+\frac{1}{t-1})}.$$

Then $M \log_B(t-1) = A$. Under this parameter choice, the total complexity is

$$\left(2 + 2\left\lceil \frac{\lfloor A \rfloor + 2}{t} \right\rceil\right) \times \mathbb{Z}[\zeta_q]\text{-BR}.$$

For $t = 64$,

$$A = \frac{2\ln 63}{\ln(1+\frac{1}{63})} \approx 526.0,$$

and thus the complexity is

$$\left(2 + 2\left\lceil \frac{528}{64} \right\rceil\right) \times \mathbb{Z}[\zeta_q]\text{-BR} = 20 \times \mathbb{Z}[\zeta_q]\text{-BR}.$$

For sufficiently large t,

$$2 + 2\left\lceil \frac{\lfloor A \rfloor + 2}{t} \right\rceil \sim 4\ln t,$$

so the asymptotic complexity is $\widetilde{O}(1) \times \mathbb{Z}[\zeta_q]$-BR. This improves over the $O(t) \times \mathbb{Z}[\zeta_q]$-BR complexity of [21].

Practical complexity. Table 2 compares the number of equivalent blind rotations required by our method and [21].

Table 2. Practical comparison of bootstrapping counts with [21].

t	B	M	Our method # $\mathbb{Z}[\zeta_q]$-BR	[21] # $\mathbb{Z}[\zeta_q]$-BR
$2^4 = 16$	100	142.70	14	80
$2^6 = 64$	2	88.03	20	320
$2^7 = 128$	100	1174.42	22	640

Applications of Non-negative Integer Division

Truncated division over general integers. For integer division, when the divisor is negative, the quotient and remainder are not uniquely determined by the condition $|r| < |d|$. For example, $-1 \div -2$ may take quotient 1 with remainder 1, or quotient 0 with remainder -1. Both satisfy the absolute remainder condition. To remove this ambiguity, we use truncated division.

Definition 2. *For any integers $m, d \in [-t, t)$ with $d \neq 0$, an integer h is the quotient of $m \div d$ if $|h|$ is minimal and $|r| < |d|$, where $r = m - dh$.*

Using Algorithm 1, we obtain the following homomorphic framework for truncated division (Algorithm 2). Its correctness is proved in Appendix B.

Algorithm 2: Truncated division IntDiv2(m, d)

Input: $\mathrm{ct}_m \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(m)$,
$\mathrm{ct}_d \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(d)$,
Bootstrapping keys: BK,
KeySwitch keys: KS.
Output: $\mathrm{ct}_h \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(h)$ where $h \in \mathbb{Z}_{2t}$ is the quotient of $m \div d$.

$\mathrm{ct}_{\mathrm{sm}} \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(\mathrm{sign}(m)t/2) \leftarrow \mathrm{FBP}(\mathrm{ct}_m, t/2 \cdot \mathrm{sign}(\cdot))$;
$\mathrm{ct}_{mm} \leftarrow (0^n, q/4) - \mathrm{ct}_{sm}$;
$\mathrm{ct}_{\mathrm{sd}} \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(\mathrm{sign}(d)t/2) \leftarrow \mathrm{FBP}(\mathrm{ct}_d, t/2 \cdot \mathrm{sign}(\cdot))$;
$\mathrm{ct}_{md} \leftarrow (0^n, q/4) - \mathrm{ct}_{sd}$;
$\mathrm{ct}_{\mathrm{bm}} \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(\mathrm{sign}(m)\bar{m}) \leftarrow \mathrm{FBP}(\mathrm{ct}_m, \mathrm{sign}(\cdot)(\cdot \bmod t))$;
$\mathrm{ct}_{\mathrm{bd}} \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(\mathrm{sign}(d)\bar{d}) \leftarrow \mathrm{FBP}(\mathrm{ct}_d, \mathrm{sign}(\cdot)(\cdot \bmod t))$;
$\mathrm{ct}_{am} = \mathrm{ct}_{bm} + \mathrm{ct}_{mm}$;
$\mathrm{ct}_{ad} = \mathrm{ct}_{bd} + \mathrm{ct}_{md}$;
$\mathrm{ct}_{ah} \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(|h|) \leftarrow \mathrm{IntDiv}(\mathrm{ct}_{am}, \mathrm{ct}_{ad})$;
$\mathrm{ct}_{mh} = \mathrm{ct}_{mm} + \mathrm{ct}_{md}$;
$\mathrm{ct}_{bh} = \mathrm{ct}_{mh} + \mathrm{ct}_{ah}$;
$\mathrm{ct}_h \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(h) \leftarrow \mathrm{FBP}(\mathrm{ct}_{bh}, \psi)$ ▷ ψ is defined in Eq. (7);
return ct_h.

Whenever the invoked function is negacyclic, such as $t/2 \cdot \mathrm{sign}(\cdot)$ and $(\cdot) \bmod t$, we use FBP rather than GFBP, since FBP has lower noise growth and lower computational cost. The overall complexity remains $\widetilde{O}(1)$ functional bootstrappings, and the noise can be made optimal.

Modular reduction. The operation $m \bmod d$ is also common. If both m and d are encrypted inputs, direct table evaluation requires a LUT of size t^2. Since our framework computes non-negative division with remainder, it can directly output the remainder

$$r = m - d\lfloor m/d \rfloor.$$

For $d \neq 0$, this value satisfies $0 \leq r < d \leq t-1$, and hence the output remains in $[t]$. Thus, the overall complexity remains $\widetilde{O}(1)$ functional bootstrappings, and the noise can be made optimal.

Other Functions via Exponential-Logarithmic Exchange For some multivariate functions, the same idea together with a scaling factor $M(t)$ can express the function as a combination of univariate functions.

Let $t \geq 2$, and consider integer pairs (a, b) with $a, b \in \{2, \ldots, t\}$. The trivial cases $a = 1$ or $b = 1$ can be handled separately. We aim to compute $\lfloor a^{1/b} \rfloor, \lfloor \log_a b \rfloor$, with asymptotic LUT complexity below $O(t^2)$. Using

$$a^{1/b} = 2^{(\log a)/b} = 2^{2^{\log\log a - \log b}}, \qquad \log_a b = \frac{\log b}{\log a} = 2^{\log\log b - \log\log a},$$

both functions can be reduced to first computing a logarithmic difference and then applying exponentiation. A similar analysis gives outer LUT complexities

$$O\big(t(\log t)^3\big), \qquad O\big(t(\log t)^2 \log\log t\big),$$

respectively. Therefore, the corresponding numbers of functional bootstrappings are

$$O\big((\log t)^3\big), \qquad O\big((\log t)^2 \log\log t\big),$$

respectively, both below $O(t)$ table evaluations.

Other elementary transformations may also help for special functions. For example, define

$$H(x) = \frac{\operatorname{sgn}(x) + 1}{2}.$$

Then

$$\max(a, b) = b + (a - b)H(a - b).$$

Thus $\max(a, b)$ can be computed using one table lookup of size t, one multiplication, and one addition, reducing the table size from t^2 to t.

4 Efficient Bootstrapping for General Multivariate Functions

We now consider how to efficiently bootstrap a general multivariate function

$$f(x_1, x_2, \ldots, x_\ell), \qquad x_i \in [t], \qquad f(x_1, \ldots, x_\ell) \in [t].$$

The first question is whether a finite discrete multivariate function can always be written as

$$f(x_1, \ldots, x_\ell) = q(p_1(x_1) + \cdots + p_\ell(x_\ell)).$$

If such a representation exists, we want the domain of the outer function q to be as small as possible. The inner functions require only ℓ table lookups of size t, while the complexity of the outer lookup depends on the range of $p_1(x_1) + \cdots + p_\ell(x_\ell)$.

4.1 A Universal Framework for Multivariate Functional Bootstrapping

Theorem 3 (Universal representation of multivariate functions). *Let $f : [t]^\ell \to [t]$ be any function. Then there exist univariate maps*

$$p_i : [t] \to \mathbb{Z} \quad (i = 1, \dots, \ell)$$

and

$$q : \mathbb{Z} \to [t]$$

such that

$$f(x_1, \dots, x_\ell) = q\big(p_1(x_1) + \cdots + p_\ell(x_\ell)\big)$$

for all $(x_1, \dots, x_\ell) \in [t]^\ell$.

Proof. Set

$$p_i(a) = at^{i-1}, \qquad a \in [t], \quad i = 1, \dots, \ell,$$

and define

$$\Phi(x_1, \dots, x_\ell) = \sum_{i=1}^{\ell} p_i(x_i) = \sum_{i=1}^{\ell} x_i t^{i-1}.$$

Then Φ is the base-t encoding of $(x_1, \dots, x_\ell)$, and hence is injective on $[t]^\ell$.

We now define $q : \mathbb{Z} \to [t]$. For every $z \in \mathrm{Im}(\Phi)$, there exists a unique $(\hat{x}_1, \dots, \hat{x}_\ell) \in [t]^\ell$ such that

$$z = \Phi(\hat{x}_1, \dots, \hat{x}_\ell).$$

For such z, define

$$q(z) = f(\hat{x}_1, \dots, \hat{x}_\ell).$$

For $z \notin \mathrm{Im}(\Phi)$, define $q(z)$ arbitrarily, for example $q(z) = 0$.

Therefore, for every $(x_1, \dots, x_\ell) \in [t]^\ell$,

$$q\big(\Phi(x_1, \dots, x_\ell)\big) = f(x_1, \dots, x_\ell),$$

that is,

$$f(x_1, \dots, x_\ell) = q\big(p_1(x_1) + \cdots + p_\ell(x_\ell)\big).$$

This proves the claim.

The theorem shows that such a decomposition always exists, but the above construction is a complete encoding and does not yield asymptotic compression.

In practice, Φ need not be injective on all inputs. We only need that if

$$f(x_1, \dots, x_\ell) \neq f(x'_1, \dots, x'_\ell),$$

then

$$\sum_{i=1}^{\ell} p_i(x_i) \neq \sum_{i=1}^{\ell} p_i(x'_i).$$

That is, different output classes must be separated by distinct sum values. Let

$$b(x_1, \ldots, x_\ell) = \sum_{i=1}^{\ell} p_i(x_i),$$

and define

$$E = \max_{x \in [t]^\ell} b(x) - \min_{x \in [t]^\ell} b(x).$$

Then the outer function q only needs a table over an interval of size $E + 1$.

Problem 1 (Minimum-span representation). Given $\ell, t \in \mathbb{N}$ and $f : [t]^\ell \to [t]$, find non-negative integer values

$$p_i(a), \qquad i = 1, \ldots, \ell, \quad a \in [t],$$

such that whenever

$$f(x_1, \ldots, x_\ell) \neq f(x'_1, \ldots, x'_\ell),$$

we have

$$\sum_{i=1}^{\ell} p_i(x_i) \neq \sum_{i=1}^{\ell} p_i(x'_i),$$

and the objective

$$E = \max_{x \in [t]^\ell} \sum_{i=1}^{\ell} p_i(x_i) - \min_{x \in [t]^\ell} \sum_{i=1}^{\ell} p_i(x_i)$$

is minimized.

When E attains its minimum, the number of bootstrappings is reduced from $t^{\ell-1}$ to

$$\ell + \left\lceil \frac{E+1}{t} \right\rceil.$$

If t is small, then several variables can be packed together to improve the utilization of the inner-function bootstrapping and further reduce the number of bootstrappings. For example, when $t = 4$, one may pack x_1, x_2, x_3 and reconstruct them as one element in [64], so that the inner functions p_1, p_2, p_3 can be evaluated by one bootstrapping. In particular, when $t = 2$, the inner functions do not require bootstrapping.

This problem always has a feasible solution, for example the encoding construction above. If f is surjective onto $[t]$, then

$$E + 1 \geq t.$$

Thus the outer table cannot be smaller than the number of output classes, but substantial compression is possible when $E \ll t^\ell$. Since this is an optimization problem, we solve it using simulated annealing.

4.2 LUT Compression via Simulated Annealing

Let $b(x_1, \ldots, x_\ell) = \sum_{i=1}^{\ell} p_i(x_i), \qquad E = \max_{[t]^\ell} b(x_1, \ldots, x_\ell) - \min_{[t]^\ell} b(x_1, \ldots, x_\ell)$. We aim to minimize E while preserving the separation constraint. In simulated annealing, each step randomly generates a candidate solution from the current solution. If the candidate still satisfies the separation constraint, it is accepted according to the energy change. We use three local moves.

Merge move. Randomly choose two inputs $(u_1, \ldots, u_\ell), (v_1, \ldots, v_\ell) \in [t]^\ell$ satisfying $f(u_1, \ldots, u_\ell) = f(v_1, \ldots, v_\ell)$, and choose a coordinate i such that $u_i \neq v_i$. Let $d = b(v_1, \ldots, v_\ell) - b(u_1, \ldots, u_\ell), \qquad b(x_1, \ldots, x_\ell) = \sum_{j=1}^{\ell} p_j(x_j)$. By increasing $p_i(u_i)$ by d, or decreasing $p_i(v_i)$ by d, the two sum values corresponding to $(u_1, \ldots, u_\ell)$ and $(v_1, \ldots, v_\ell)$ can be merged. This move attempts to reduce the dispersion of sum values within the same output class.

Aggregation move. Randomly choose two inputs $(u_1, \ldots, u_\ell), (v_1, \ldots, v_\ell) \in [t]^\ell$ satisfying $f(u_1, \ldots, u_\ell) = f(v_1, \ldots, v_\ell)$, and choose a coordinate i such that $u_i \neq v_i$. If $d = b(v_1, \ldots, v_\ell) - b(u_1, \ldots, u_\ell)$ is even, update $p_i(u_i) \leftarrow p_i(u_i) + \frac{d}{2}, \qquad p_i(v_i) \leftarrow p_i(v_i) - \frac{d}{2}$. This moves the two corresponding sum values toward the midpoint, thereby compressing the overall span more smoothly.

Perturbation move. Randomly choose a table entry $p_i(a)$, where $i \in \{1, \ldots, \ell\}, \qquad a \in [t]$, and update $p_i(a) \leftarrow p_i(a) + \Delta, \qquad \Delta \in \{-2, -1, 1, 2\}$. This operation makes small local adjustments and helps the search escape local optima.

For a multivariate function $f(x_1, \ldots, x_\ell)$, the simulated-annealing compression algorithm is as follows.

Algorithm 3: Simulated-annealing compression for multivariate-function lookup tables

Input: $\ell, t \in \mathbb{N}$, a function $f : [t]^\ell \to [t]$;
maximum number of iterations M, initial temperature T_0, cooling rate $c \in (0, 1)$;
reheating interval R, reheating factor $\beta \in (0, 1]$.
Output: Univariate functions $p_i(j)$, $i = 1, \ldots, \ell$, $j \in [t]$, and the minimum span $E_{\min}$.

Initialize

$$p_i(j) \leftarrow j \cdot t^{i-1},$$

so that $b(x_1, \ldots, x_\ell) = \sum_i p_i(x_i)$ is injective on different inputs x;
Compute the initial span

$$E \leftarrow \max_{[t]^\ell} b(x_1, \ldots, x_\ell) - \min_{[t]^\ell} b(x_1, \ldots, x_\ell),$$

and set $E_{\min} \leftarrow E$, $T \leftarrow T_0$, $cnt \leftarrow 0$, where cnt records the number of consecutive iterations without accepting an improved or feasible candidate;
for iter $= 1$ **to** M **do**
  Randomly perform one merge, aggregation, or perturbation move to obtain a candidate solution p_i';
  if p_i' *violates the separation constraint* **then**
    Discard the candidate and continue to the next iteration;
  end
  Compute the candidate span E_{new}, and set $\Delta E \leftarrow E_{\text{new}} - E$;
  if $E_{\text{new}} < E_{\min}$ **then**
    Record the candidate as the best solution and set $E_{\min} \leftarrow E_{\text{new}}$;
    Accept p_i' as the current solution, set $E \leftarrow E_{\text{new}}$, and set $cnt \leftarrow 0$;
  end
  else
    Accept p_i' as the current solution with probability $\exp(-\Delta E/T)$;
    if *accepted* **then**
      Set $E \leftarrow E_{\text{new}}$, $cnt \leftarrow 0$;
    end
    else
      Set $cnt \leftarrow cnt + 1$;
    end
  end
  Set $T \leftarrow T \cdot c$;
  if $cnt \geq R$ **then**
    Set $T \leftarrow T + (T_0 - T) \cdot \beta$, and set $cnt \leftarrow 0$;
  end
end
return the best $p_i(j)$ and $E_{\min}$;

Remark. When t^ℓ does not exceed the experimental threshold, which is about 2^{14} in our implementation, all input points can be exhaustively checked so that the output solution strictly satisfies the separation constraint. For larger scales, random sampling can be used to check the constraint, trading strict verification

for efficiency. In practice, the simulated-annealing procedure can be repeated multiple times to obtain a smaller value of E. Moreover, an initial solution with a larger span may sometimes lead to a better final solution with a smaller optimized span.

4.3 Compression Results via Simulated Annealing

We now present compression results for several common multivariate functions. In the following tables, we report the total size of the inner and outer lookup tables. This quantity reflects the storage cost of the decomposed representation and is also closely related to the functional-bootstrapping time, since larger lookup tables generally require larger bootstrapping domains or more bootstrapping calls.

For [3], the lookup-table size is estimated according to the parameter selection in Table 1 of that work: choose $\lambda + 2a$ as small as possible subject to $1.05 \cdot t^{\ell} < (a+1)(\ell+\lambda)$, and let each functional-bootstrapping lookup table have size p, where p is the smallest prime greater than t. The total number of records is then $(\lambda + 2a) \cdot p$. For our method, the total number of records is $\ell t + E + 1$.

Our local experiments show that this method is particularly effective for the following types of functions.

Frequency-type functions. These functions are mainly determined by the frequencies of symbols appearing in the input. We consider two representative examples: the mode function and the median function.

Mode function. Let

$$f_{\mathrm{mode}}(\mathbf{x}) = \mathrm{mode}(x_1, \ldots, x_\ell),$$

where, if several values occur with the same maximum frequency, the smallest one is chosen as the mode.

Median function. Let

$$f_{\mathrm{med}}(\mathbf{x}) = \mathrm{med}(x_1, \ldots, x_\ell),$$

where the median is taken according to the natural order $0 < 1 < \cdots < t-1$, and the lower median is used when ℓ is even.

The compression results for both frequency-type functions are shown in Table 3. Since the two functions are tested under the same parameter settings, we combine their results into a single table.

Table 3. Compression results for frequency-type functions.

ℓ	t	t^ℓ	Mode		Median	
			Our method	[3]	Our method	[3]
10	2	1024	52	243	55	243
14	2	16384	103	1065	88	1065
5	3	243	78	195	71	195
6	3	729	127	355	61	355
3	4	64	76	95	58	95

Threshold-type functions. These functions are determined by whether a statistic exceeds a prescribed threshold. We consider the following general symbol-set threshold function. For parameters $A \subseteq [t]$ and $\tau \in \{0, \ldots, \ell\}$, define

$$f_{A,\tau}(x_1, \ldots, x_\ell) = \begin{cases} 1, & \#\{i \in \{1, \ldots, \ell\} : x_i \in A\} \geq \tau, \\ 0, & \text{otherwise.} \end{cases}$$

The experiments below instantiate this family with different choices of ℓ, t, A, and τ. The compression results are shown in Table 4.

Table 4. Total LUT sizes for general symbol-set threshold functions.

ℓ	t	A	τ	t^ℓ	Our method	[3]
8	2	$\{1\}$	4	256	33	111
5	3	$\{1\}$	2	243	21	195
6	3	$\{0, 2\}$	3	729	34	355
6	4	$\{0, 1, 2\}$	3	4096	59	890
8	3	$\{1\}$	4	6561	57	1125

Interval-type functions. These functions are determined by whether a statistic lies in a prescribed interval. We give two representative cases: the Hamming-weight interval function and the general symbol-set interval function.

Hamming-weight interval function. Let the input be $\mathbf{x} \in [2]^\ell$, and define

$$f_{[a,b]}(\mathbf{x}) = \mathbf{1}\big[a \leq c_{\{1\}}(\mathbf{x}) \leq b\big],$$

where $0 \leq a \leq b \leq \ell$. This function only depends on whether the number of ones in the input lies in a specified interval. Since this is a binary-input special case, the count $c_{\{1\}}(\mathbf{x})$ can be obtained by homomorphic additions, and the experiment mainly demonstrates the compression effect of the outer LUT. The compression results are shown in Table 5.

Table 5. Total LUT sizes for Hamming-weight interval functions.

ℓ	t	$[a,b]$	t^ℓ	Our method	[3]
5	2	$[1,3]$	32	16	30
6	2	$[2,4]$	64	24	48
8	2	$[3,5]$	256	25	111
10	2	$[4,6]$	1024	31	243
12	2	$[5,7]$	4096	37	516

General symbol-set interval function. Let $A \subseteq [t]$, and define

$$g_{A,[a,b]}(\mathbf{x}) = \mathbf{1}[a \le c_A(\mathbf{x}) \le b]\,,$$

where $\mathbf{x} \in [t]^\ell$ and $0 \le a \le b \le \ell$. This function checks whether the number of input symbols lying in A falls into a specified interval. Unlike the binary Hamming-weight case above, the indicator $\mathbf{1}[x_i \in A]$ is generally nonlinear for $t > 2$, and thus typically requires inner LUT evaluations. The compression results are shown in Table 6.

Table 6. Total LUT sizes for non-binary general symbol-set interval functions.

ℓ	t	A	$[a,b]$	t^ℓ	Our method	[3]
6	3	$\{1\}$	$[2,4]$	729	25	355
6	4	$\{1,3\}$	$[2,4]$	4096	31	890

A counterexample outside the above structures. We also give a function that does not belong to the above low-complexity structures. Its output is determined by nonlinear operations on multiple variables, so it is generally harder to compress strongly using the proposed decomposition.

Bit extraction from a discretized activation function. Let $\mathbf{x} \in [2]^\ell$, and interpret it as an integer

$$X = \sum_{i=1}^{\ell} x_i 2^{\ell-i}.$$

Define the discretized activation transform

$$\phi_{\mathrm{act}}(X) = \Big(\big((X \cdot 131 + 17) \oplus ((X \gg 3) + (X \ll 5))\big) \bmod 2^\ell\Big),$$

where $\oplus$ denotes bitwise XOR, and $\gg$ and $\ll$ denote logical right and left shifts, respectively. For bit index $k = \ell - 2$, define

$$f_{\mathrm{act},k}(\mathbf{x}) = (\phi_{\mathrm{act}}(X) \gg k)\ \&\ 1.$$

Here &1 denotes the bitwise AND with 1, which extracts the least significant bit after the right shift. Equivalently, $f_{\mathrm{act},k}$ extracts the $(k+1)$-th bit of $\phi_{\mathrm{act}}(X)$ when bits are counted from right to left, starting from the least significant bit.

Table 7. Total LUT sizes for bit extraction from a discretized activation function modulo 2^{ℓ}.

ℓ	T	k	2^{ℓ}	Our method	[3]
8	256	6	256	68	111
10	1024	8	1024	972	243
12	4096	10	4096	4104	516

Functional-bootstrapping algorithm from the decomposition. Finally, based on the above decomposition, we give the homomorphic algorithm for evaluating a multivariate function. Let $f(x_1, \ldots, x_\ell) \in [t]$, with $x_i \in [t]$.

Algorithm 4: Homomorphic evaluation of a multivariate function $f(x_1, \ldots, x_\ell)$

Input: $\mathrm{ct}_{x_i} \in \mathrm{LWE}_{s,q/t}(x_i)$ for $i \in [\ell]$; bootstrapping key BK, key-switching key KS.

Output: $\mathrm{ct}_f \in \mathrm{LWE}_{s,q/2t}(c)$, where $c = f(x_1, \ldots, x_\ell) \in [t]$.

1 For each $i \in [\ell]$, compute $\mathrm{ct}_{p_i(x_i)} \leftarrow \mathrm{GFBP}(\mathrm{ct}_{x_i}, p_i)$;
2 Compute $\mathrm{ct}_b \leftarrow \sum_{i=1}^{\ell} \mathrm{ct}_{p_i(x_i)}$;
3 Compute $\mathrm{ct}_f \leftarrow \mathrm{GFBP}(\mathrm{ct}_b, q)$;
4 **return** ct_f;

Remark. When $t = 2$, the first step does not require functional bootstrapping in many cases, since each inner function $p_i(x_i)$ can be represented by an affine function of the input bit x_i and hence computed by homomorphic linear operations. Moreover, when t is small, several inner LUT evaluations can be packed into a single bootstrapping call, thereby reducing the actual number of bootstrappings required in implementation.

5 Experiments

Experimental environment. All experiments were run on a desktop machine equipped with an Intel(R) Core(TM) i7-14700 processor with 20 cores and 28 threads, a reported base frequency of 2.10 GHz, and 24 GB RAM. The host operating system was Microsoft Windows 11 Home Chinese Edition 64-bit, version 10.0.26200. Bootstrapping-time experiments were performed in Ubuntu 24.04.3 LTS under Windows Subsystem for Linux 2 (WSL2), with Linux kernel version 6.6.87.2-microsoft-standard-WSL2. The simulated-annealing compression algorithm was implemented in Python 3.9. Homomorphic bootstrapping experiments were implemented using Rust 1.96.0, Cargo 1.96.0, and TFHE-rs 0.8.7. Unless otherwise stated, all reported times are average online computation times.

We emphasize that the general functional bootstrapping technique of [26], which is used in our theoretical complexity analysis, does not currently provide a publicly available implementation. Hence, our timing experiments are implemented with publicly available TFHE-rs primitives. The reported times therefore reflect our current prototype implementation, while the theoretical complexity estimates follow the cost model of [26] and related functional-bootstrapping methods.

5.1 Bootstrapping Implementation of Non-negative Division with Remainder

Consider non-negative integer division with remainder

$$g(m, d) = \lfloor m/d \rfloor,$$

with domain $[t] \times ([t] \setminus \{0\})$. Our method decomposes this bivariate LUT into two univariate logarithmic functional bootstrappings, one homomorphic subtraction, and one outer exponential functional bootstrapping. According to the complexity analysis in Section 3.1.1, under theoretically optimal parameters, the total equivalent cost is

$$\left(2 + 2\left\lceil \frac{\lfloor A \rfloor + 1}{t} \right\rceil\right) \times \mathbb{Z}[\zeta_q]\text{-BR}, \qquad A = \frac{2\ln(t-1)}{\ln(1 + \frac{1}{t-1})}.$$

Table 8 compares the number of lookup-table records among our method, [3], and [21]. For [3], we use the same estimation method as in Section 4: choose $\lambda+2a$ as small as possible subject to

$$1.05 \cdot t^{\ell} < (a+1)(\ell + \lambda),$$

and let each PBS lookup table have size equal to the smallest prime p greater than t. Thus the record count is $(\lambda + 2a)p$. For [21], their Section 4.4 states that $N\tau$ torus values must be stored for the division-by-constant test vectors; using their experimental parameter $N = 1024$ and $\tau = t + 1$, this gives $1024(t + 1)$ records. Our method reduces the number of records in all three parameter settings.

Table 8. Lookup-table compression for division with remainder.

ℓ	t	LUT size of [3]	LUT size of [21]	LUT size of our method
2	16	731	17408	202
2	32	3293	33792	498
2	64	12194	66560	1280

Table 9 compares the bootstrapping running time between our method , [3], and [21].

Table 9. Running-time comparison for bootstrapping division with remainder.

ℓ	t	Estimated time of [3] (ms)	Estimated time of [21] (ms)	Time of our method (ms)
2	16	80.10	930.00	267.02
2	32	374.20	1860.00	305.17
2	64	1369.37	3720.00	381.46

For [21], the original paper reports 0.93 s for 4-bit division, corresponding to $t = 16$, and its Algorithm 10 enumerates all possible divisors, so the cost scales linearly with t. Hence we use 930 ms for $t = 16$ and linearly estimate 1860 ms and 3720 ms for $t = 32$ and $t = 64$, respectively.

For [3], we estimate the running time from the timing tables reported in the appendix of [3]. Specifically, we first compute the decomposition parameters by the same rule used for Table 8, namely by minimizing $\lambda+2a$ under the constraint $1.05 \cdot t^{\ell} < (a+1)(\ell+\lambda)$. We then multiply the resulting number of PBS calls by the corresponding per-PBS timing reported in the appendix of [3], using the closest matching plaintext modulus.

Tables 9 show that the main advantage of our method first appears in structural compression and asymptotic complexity. For small parameters, the constants introduced by theoretically optimal parameter selection remain non-negligible, so when $t = 16$, the running time is not better than the estimate for [3]. However, when t grows to 32 and 64, our method grows much more slowly.

5.2 Bootstrapping Results for General Multivariate Functions

This section evaluates the general multivariate-function decomposition framework from Section 4. The record-count results have already been reported in the tables in Section 4.

Table 10 reports online end-to-end computation times in the current TFHE-rs prototype implementation. For each function, the online evaluation was repeated at least 30 times on independently sampled inputs. Timing starts after the input ciphertexts have been generated and ends after the outer functional bootstrapping outputs the result ciphertext. The online computation includes

$$\mathrm{PBS}(p_1), \ldots, \mathrm{PBS}(p_\ell), \qquad \sum_i p_i(x_i) \text{ by homomorphic additions}, \qquad \mathrm{PBS}(q).$$

For [3], the estimated running times are obtained from the timing tables reported in the appendix of [3]. For each function, we use the decomposition shape and record-count estimation described in Section 4 to determine the corresponding number and size of PBS calls. We then scale the estimate using the reported timing for the closest available LUT bitwidth in the appendix of [3].

Table 10. Online end-to-end computation times for general multivariate functions.

Function	Parameters	Estimated time of [3] (ms)	Our time (ms)
Hamming-weight interval function	$t = 2$, $\ell = 10$ $[a, b] = [4, 6]$	198.6	102.7
General symbol-set interval function	$t = 4$, $\ell = 6$ $A = \{0, 2\}$, $\tau = 3$	416.9	312.4
General symbol-set threshold function	$t = 4$, $\ell = 6$ $A = \{0, 2\}$, $\tau = 3$	416.9	304.1
Median function	$t = 3$, $\ell = 6$	141.5	309.9

6 Conclusion

This paper proposed a multivariate functional-bootstrapping framework for TFHE. The core idea is to reduce the number of records and the bootstrapping cost required by direct multivariate LUT evaluation through variable separation and outer-LUT compression.

First, for non-negative integer division with remainder

$$g(m, d) = \left\lfloor \frac{m}{d} \right\rfloor,$$

we used a logarithmic transformation to decompose the bivariate function into two univariate logarithmic PBS calls, one homomorphic subtraction, and one outer exponential PBS call. By introducing the rounded logarithmic function $\mathrm{clog}_{B,M}$ and scaling factor M, we proved that the method exactly recovers the quotient. The equivalent complexity is

$$\left(2 + 2\left\lceil \frac{\lfloor M \log_B(t-1) \rfloor + 1}{t} \right\rceil\right) \times \mathbb{Z}[\zeta_q]\text{-BR},$$

which becomes $\widetilde{O}(1)$ $\mathbb{Z}[\zeta_q]$-BR under theoretically optimal parameters. Based on this non-negative division algorithm, we further gave homomorphic frameworks for modular reduction and truncated division over general integers.

Second, we extended the idea to general multivariate functions. We proved that every finite function

$$f : [t]^\ell \to [t]$$

can be written as

$$f(x_1, \ldots, x_\ell) = q\left(\sum_{i=1}^{\ell} p_i(x_i)\right).$$

In practical compression, it is enough to ensure that different output classes correspond to separated sum values. Therefore, the outer LUT size is determined by the span $E + 1$. We then used simulated annealing with reheating to search

for inner functions p_i with small span, thereby compressing the total number of LUT records.

Experimental results show that the advantage of our method mainly lies in structural compression and the running-time trend for medium-size parameters. For non-negative integer division with remainder, our method outperforms the estimated time of [3] for $t = 32$ and $t = 64$. For general multivariate functions, our method reduces the total number of inner and outer LUT records for statistically aggregating functions such as interval functions, and achieves lower online end-to-end computation time in most experiments.

Acknowledgments. We acknowledge the assistance of ChatGPT in improving the manuscript presentation and in drafting auxiliary code used during the preparation of this work. All AI-assisted content, including textual suggestions and code, was reviewed, verified, and revised by the authors.

References

1. Akherati, S., Cai, J., Zhang, X.: Efficient generalized integer division and modular reduction architectures for homomorphic encryption. Journal of Signal Processing Systems **97**(2–4), 157–171 (2025)
2. Bergerat, L., Boudi, A., Bourgerie, Q., Chillotti, I., Ligier, D., Orfila, J.-B., Tap, S.: Parameter optimization and larger precision for (T)FHE. Journal of Cryptology **36**(3), 28 (2023)
3. Belaïd, S., Bon, N., Rivain, M.: Decomposition of large look-up tables for fast homomorphic evaluation. IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES 2026). Cryptology ePrint Archive, Report 2026/724 (2026). https://eprint.iacr.org/2026/724
4. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: Fully homomorphic encryption without bootstrapping. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference, pp. 309–325. ACM (2012)
5. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Advances in Cryptology – CRYPTO 2012, LNCS, vol. 7417, pp. 868–886. Springer, Heidelberg (2012)
6. Chen, J., Feng, Y., Liu, Y., Wu, W., et al.: Faster binary arithmetic operations on encrypted integers. In: The 7th International Workshop on Computer Science and Engineering, pp. 1–2. IEEE (2017)
7. Chen, Y., Gong, G.: Integer arithmetic over ciphertext and homomorphic data aggregation. In: 2015 IEEE Conference on Communications and Network Security, pp. 628–632. IEEE (2015)
8. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. Cryptology ePrint Archive, Report 2016/870 (2016)
9. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In: Advances in Cryptology – ASIACRYPT 2020, LNCS, vol. 12491, pp. 670–699. Springer, Heidelberg (2020)
10. Carpov, S., Izabachène, M., Mollimard, V.: New techniques for multi-value input homomorphic evaluation and applications. In: Matsui, M. (ed.) CT-RSA 2019, LNCS, vol. 11405, pp. 106–126. Springer, Cham (2019)

11. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology – ASIACRYPT 2017, LNCS, vol. 10624, pp. 409–437. Springer, Heidelberg (2017)
12. Ducas, L., Micciancio, D.: FHEW: Bootstrapping homomorphic encryption in less than a second. In: Advances in Cryptology – EUROCRYPT 2015, LNCS, vol. 9056, pp. 617–640. Springer, Heidelberg (2015)
13. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144 (2012)
14. Guimaraes, A., Borin, E., Aranha, D.F.: Revisiting the functional bootstrap in TFHE. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(2), 229–253 (2021)
15. Gentry, C.: A fully homomorphic encryption scheme. Ph.D. thesis, Stanford University (2009)
16. Halevi, S., Shoup, V.: Design and implementation of a homomorphic encryption library. IBM Research Report RC25779 (2013)
17. Ma, S., Huang, T., Wang, A., Zhou, Q., Wang, X.: Fast and accurate: Efficient full-domain functional bootstrap and digit decomposition for homomorphic computation. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(1), 592–616 (2024)
18. Morimura, K., Maeda, D., Nishide, T.: Accelerating polynomial evaluation for integer-wise homomorphic comparison and division. Journal of Information Processing **31**, 288–298 (2023)
19. Maeda, D., Morimura, K., Narisada, S., Fukushima, K., Nishide, T.: Efficient homomorphic evaluation of arbitrary uni/bivariate integer functions and their applications. IEICE Transactions on Fundamentals **E107.A**(3), 234–247 (2024)
20. Okada, H., Cid, C., Hidano, S., Kiyomoto, S.: Linear depth integer-wise homomorphic division. In: Information Security Theory and Practice, LNCS, vol. 11604, pp. 91–106. Springer, Heidelberg (2019)
21. Okada, H., Kiyomoto, S., Cid, C.: Integer-wise functional bootstrapping on TFHE: Applications in secure integer arithmetics. Information **12**(8), 342 (2021)
22. Prantl, T., Horn, L., Engel, S., Ifflander, L., Beierlieb, L., Krupitzer, C., Bauer, A., Sakarvadia, M., Foster, I., Kounev, S.: De bello homomorphico: Investigation of the extensibility of the OpenFHE library with basic mathematical functions. International Journal of Information Security **23**(2), 1149–1169 (2023)
23. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM **56**(6), 1–40 (2009)
24. Trama, D., Clet, P.-E., Boudguiga, A., Sirdey, R.: A homomorphic AES evaluation in less than 30 seconds by means of TFHE. In: Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography, pp. 79–90. ACM (2023)
25. Xu, C., Chen, J., Wu, W., Feng, Y.: Homomorphically encrypted arithmetic operations over the integer ring. In: Information Security Practice and Experience, LNCS, vol. 10060, pp. 249–264. Springer, Heidelberg (2016)
26. Xia, H., Liu, F.-H., Wang, H.: More efficient functional bootstrapping for general functions in polynomial modulus. In: Theory of Cryptography – TCC 2024, LNCS, vol. 15367, pp. 130–163. Springer, Cham (2024)

A Basic Framework of the BBR26 Scheme

The large-LUT decomposition method of [3] is as follows.

First, they introduce the definition of atomic functions. In our notation, for an input tuple $(x_1, \ldots, x_\ell)$, a function of the form

$$\phi(x_1, \ldots, x_\ell) = \psi\left(\sum_{j=1}^{r} \alpha_j x_j\right)$$

is called an atomic function, where ψ is an arbitrary univariate function over the embedding field. Atomic functions are used to generate auxiliary variables that are linearly independent from the original inputs. More precisely, starting from the original inputs $x_1, \ldots, x_\ell$, an atomic-function chain generates

$$x_{\ell+1}, \ldots, x_{\ell+\lambda},$$

where each new variable $x_{\ell+h}$ is obtained from the variables already available:

$$x_{\ell+h} = \psi_h\left(\sum_{j=1}^{\ell+h-1} \alpha_{h,j} x_j\right), \qquad h = 1, \ldots, \lambda.$$

For a function $f(x_1, \ldots, x_\ell)$, since inversions of some integers are later required, $\mathbb{Z}_t$ is embedded into a finite field $\mathbb{F}_p$, where p is a prime slightly larger than t. We denote the embedded version of $f(x_1, \ldots, x_\ell)$ by $\widetilde{f}(x_1, \ldots, x_\ell)$.

They seek a representation of the form

$$\widetilde{f}(x_1, \ldots, x_\ell) = \sum_{i=0}^{a-1} \left(\sum_{j=1}^{\ell+\lambda} \beta_{i,j} x_j\right)\left(\sum_{k=1}^{\ell+\lambda} d_{i,k} x_k\right) + \sum_{j=1}^{\ell+\lambda} \beta_{a,j} x_j,$$

where the extended variable set

$$\{x_j\}_{j=1,\ldots,\ell+\lambda}$$

contains both the original inputs $x_1, \ldots, x_\ell$ and the auxiliary variables $x_{\ell+1}, \ldots, x_{\ell+\lambda}$ generated by the atomic-function chain. Here j and k are independent summation indices over the same extended variable set.

The decomposition of $\widetilde{f}(x_1, \ldots, x_\ell)$ is obtained as follows. The atomic functions and the coefficients $d_{i,k}$ are sampled uniformly at random. For a fixed sample, the LUT values corresponding to $\widetilde{f}(x_1, \ldots, x_\ell)$ and the input tuples $(x_1, \ldots, x_\ell)$ are arranged in lexicographic order, and one checks whether the resulting linear system has a solution. If it does, the coefficients $\beta_{i,j}$ are obtained, and hence the decomposition is obtained.

The work [3] argues that when

$$1.05 \cdot t^\ell < (a+1)(\ell+\lambda),$$

repeated sampling yields a decomposition with high probability.

The expression contains a product terms. The original paper computes multiplication using

$$x \cdot y = [4^{-1}]_p\big((x+y)^2 - (x-y)^2\big),$$

so one multiplication corresponds to two functional bootstrappings. In addition, computing the atomic-function chain requires λ functional bootstrappings. Therefore, the total complexity is $2a + \lambda$ functional bootstrappings. Each functional bootstrapping uses a lookup table of size p. Thus, the original table of size t^ℓ is replaced by $2a + \lambda$ lookup tables, and the total record count becomes $p(2a + \lambda)$.

B Correctness Proof for Homomorphic Truncated Division

Homomorphic truncated division is computed by Algorithm 2. Throughout this section, we assume that t is an even integer. All plaintext operations are performed in $\mathbb{Z}_{2t}$. For a ciphertext ct, let $\mu(\mathrm{ct})$ denote its plaintext. The canonical signed representative of elements in $\mathbb{Z}_{2t}$ is chosen from $\mathbb{Z}_{2t} = \{-t, \ldots, 0, \ldots, t-1\}$.

Theorem 4. *Let* $\mathrm{ct}_m \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(m)$ *and* $\mathrm{ct}_d \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(d)$*, where*

$$m, d \in \mathbb{Z} \cap (-t, t), \qquad d \neq 0, \qquad |m| \geq |d|.$$

Then Algorithm 2 outputs a ciphertext

$$\mathrm{ct}_h \in \mathrm{LWE}_{\boldsymbol{s},q/(2t)}(h),$$

where h *is the truncated quotient of* $m \div d$*.*

Proof. It suffices to trace the plaintext values. For any $x \in (-t, t)$, let $\bar{x} = x \bmod t \in [0, t)$ be its residue. We first establish the identity:

$$|x| \equiv \mathrm{sign}(x)\bar{x} + \frac{1 - \mathrm{sign}(x)}{2} t \mod 2t. \tag{1}$$

Indeed, if $x \geq 0$, then $\mathrm{sign}(x) = 1$, $\bar{x} = x$, and (1) reduces to $x = |x|$. If $x < 0$, then $\mathrm{sign}(x) = -1$, $\bar{x} = t + x$, and the right-hand side of (1) becomes $-(t + x) + t = -x = |x|$.

From the definition of FBP and the fact that $(0^n, q/4)$ encodes $t/2$, we have:

$$\mu(\mathrm{ct}_{sm}) = \frac{\mathrm{sign}(m)t}{2}, \qquad \mu(\mathrm{ct}_{sd}) = \frac{\mathrm{sign}(d)t}{2},$$

$$\mu(\mathrm{ct}_{mm}) = \frac{t}{2} - \mu(\mathrm{ct}_{sm}) = \frac{1 - \mathrm{sign}(m)}{2} t,$$

$$\mu(\mathrm{ct}_{md}) = \frac{t}{2} - \mu(\mathrm{ct}_{sd}) = \frac{1 - \mathrm{sign}(d)}{2} t.$$

The residue-sign FBPs yield:

$$\mu(\mathrm{ct}_{bm}) = \mathrm{sign}(m)\bar{m}, \qquad \mu(\mathrm{ct}_{bd}) = \mathrm{sign}(d)\bar{d}.$$

Substituting these into (1), the reconstructed absolute values satisfy:

$$\mu(\mathrm{ct}_{am}) = \mu(\mathrm{ct}_{bm}) + \mu(\mathrm{ct}_{mm}) \equiv |m| \mod 2t,$$

$$\mu(\mathrm{ct}_{ad}) = \mu(\mathrm{ct}_{bd}) + \mu(\mathrm{ct}_{md}) \equiv |d| \mod 2t.$$

Since $m, d \in (-t, t)$, we have $|m|, |d| \in [0, t)$.

Let $h = \mathrm{trunc}(m/d)$. Since $|m| \geq |d| \geq 1$, we have $h \neq 0$ and:

$$\mathrm{sign}(h) = \mathrm{sign}(m)\,\mathrm{sign}(d), \qquad |h| = \left\lfloor \frac{|m|}{|d|} \right\rfloor < t. \tag{2}$$

The subroutine IntDiv operates on non-negative inputs $|m|$ and $|d|$ to compute $[|m|/|d|]$, yielding:

$$\mu(\mathrm{ct}_{ah}) = \left\lfloor \frac{|m|}{|d|} \right\rfloor = |h|. \tag{3}$$

Next, we accumulate the sign components:

$$\mu(\mathrm{ct}_{mh}) = \mu(\mathrm{ct}_{mm}) + \mu(\mathrm{ct}_{md}) = \frac{1 - \mathrm{sign}(m)}{2}t + \frac{1 - \mathrm{sign}(d)}{2}t.$$

If $\mathrm{sign}(m) = \mathrm{sign}(d)$, then $h > 0$, and $\mu(\mathrm{ct}_{mh}) \in \{0, 2t\} \equiv 0 \mod 2t$. If $\mathrm{sign}(m) \neq \mathrm{sign}(d)$, then $h < 0$, and $\mu(\mathrm{ct}_{mh}) = t \mod 2t$. Thus:

$$\mu(\mathrm{ct}_{mh}) \equiv \begin{cases} 0 & \text{if } h > 0, \\ t & \text{if } h < 0 \end{cases} \mod 2t, \tag{4}$$

which implies the plaintext of $\mathrm{ct}_{bh} = \mathrm{ct}_{mh} + \mathrm{ct}_{ah}$ is:

$$\mu(\mathrm{ct}_{bh}) \equiv \begin{cases} |h| & \text{if } h > 0, \\ t + |h| & \text{if } h < 0 \end{cases} \mod 2t. \tag{5}$$

Since $|h| < t$, we represent $\mu(\mathrm{ct}_{bh}) \in \mathbb{Z}_{2t}$ by its signed representative y:

$$y = \begin{cases} |h| = h \in [0, t) & \text{if } h > 0, \\ (t + |h|) - 2t = -h - t \in [-t, 0) & \text{if } h < 0. \end{cases} \tag{6}$$

Now, we define the conversion function $\psi : \mathbb{Z}_{2t} \to \mathbb{Z}_{2t}$ as:

$$\psi(y) = \begin{cases} y & \text{if } y \in [0, t), \\ -y - t & \text{if } y \in [-t, 0). \end{cases} \tag{7}$$

Applying ψ to the signed representative y in (6) yields:

- If $h > 0$: $\psi(y) = y = h$.
- If $h < 0$: $\psi(y) = -y - t = -(-h - t) - t = h$.

Hence, evaluating ψ via FBP in the final step outputs $\mu(\mathrm{ct}_h) = h$.